Prospect Tracker (v1) — Local-First CRUD System

Audience	Technical reviewers
Scope	System design documentation
Prepared by	Jhondel Mellomida
Last updated	22/9/2026

1. Overview

1.1 Problem statement (why build vs. spreadsheet)

This tracking system is meant to track client acquisition and what's the progress, whether its worth continuing or not. The tracking system is meant to help simplify the process and improve the setup for such task.

Rather than using a spreadsheet for tracking, it was decided to create a system to practice coding from scratch with AI as only guide and not code generator. It's a learning approach therefore were building. Another advantage is the flexibility of it based on what we want to achieve.

1.2 Goals / Non-goals

- **Goals:** add something like *"Serve as a learning exercise for cache-aside invalidation patterns, not just a CRUD app"* if that's genuinely part of why you're building it. It's a real goal and it explains ADR-0003 later.

- **Non-goals:** add *"Fine-grained/partial cache invalidation (v1 clears the whole cache on any write)"* and *"Distributed caching (Redis deferred to v2)"* — this is what let you defend the "clear everything" decision in the ADR instead of it looking like an oversight.

2. Requirements

2.1 Functional requirements

ID	Requirement	Priority
FR-1.1	User can create an application record with company, contact(email/number), date created, last date contacted, technical audit (free text), source, status. Company and status are required	Must
FR-1.2	User can edit existing application record	Must
FR-1.3	User can delete existing application record	Must
FR-1.4	User can see individual application record	Must
FR-1.5	User can page through records	Must
FR-2.1	User can add notes along an application record like client response. Notes are auto timestamped	Must
FR-2.2	User can edit notes along an application record like client response	Must
FR-2.3	User can delete notes along an application record like client response	Must
FR-3.1	User can filter based on status	Must
FR-3.2	User can sort records by last contacted date, ascending or descending	Must
FR-3.3	User can search by company and source (case insensitive, partial match)	Must
FR-4.1	User can set a follow-up reminder (date, or offset like "3 days after last contact"); records due today or earlier are flagged in the list	Should
FR-5.1	User can export records to CSV	Could
FR-5.2	User can import records from CSV	Could

FR-6.1	Login, accounts, and profile management	Wont
FR-7.1	System writes an audit log entry (entity, action, old/new values, timestamp) on every create, update, or delete of a record or note	Must
FR-8.1	System pass telemetry data for monitoring and analysis	Wont

2.2 Non-functional requirements

ID	Category	Requirement
NFR-1	Performance	List endpoint p95 in <500ms at up to 1,000 records
NFR-2	Performance	Search endpoint p95 <500ms upon clicking button
NFR-3	Data Integrity	Deleting a record cascade to its notes and follow-up schedule; no orphans. Audit logs are excluded and survive deletion
NFR-4	Reliability	Schema changes only via Django migrations (makemigrations/migrate); every migration is reversible
NFR-5	Auditability	Audit entries are timestamped (UTC) and append-only (no edit or delete endpoint)
NFR-6	Security	API bound to localhost or a private network; v1 is single user with no auth
NFR-7	Maintainability	Backend test coverage >= 80% on view/viewset of Django
NFR-8	Maintainability	Code passes ruff (PEP 8) in CI; source files ≤ 200 lines unless justified in a comment
NFR-9	Performance	GET /records list is cached in-process (dict, TTL 30s), invalidated on any write to records or notes
NFR-10	Performance	List endpoint response time target (NFR-1) applies with and without cache hit — i.e., cache is an optimization, not a requirement for meeting the target

3. System Interface Definition

REST was used as the main architecture style for its simplicity for this project since project is mostly CRUD and monolithic. Another alternative would be GraphQL however changing the architecture style means changing the endpoints.

3.1 API endpoints

Records

Method	Path	Purpose	Satisfies	Success	Errors
POST	/api/v1/records	Add application record	FR-1.1	201	400
PATCH	/api/v1/records/{record_id}	Edit application record	FR-1.2, FR-4.1	200	404, 400

DELETE	/api/v1/records/{record_id}	Delete application record	FR-1.3	204	404
GET	/api/v1/records/{record_id}	Get one application record with its notes	FR-1.4	200	404
GET	/api/v1/records	List records (filter, sort, search, due)	FR-1.2, 3.1, 3.2, 3.3, 4.1	200	400
GET	/api/v1/records/export	Download all records into CSV	FR-5.1	200	none
POST	/api/v1/records/import	Upload CSV to records	FR-5.2	201	400, 422

Notes

Method	Path	Purpose	Satisfies	Success	Errors
POST	/api/v1/records/{record_id}/notes	Add a note to a record	FR-2.1	201	404, 400
PATCH	/api/v1/notes/{note_id}	Edit a note	FR-2.2	200	404, 400
DELETE	/api/v1/notes/{note_id}	Delete a note	FR-2.3	204	404

Audit

Method	Path	Purpose	Satisfies	Success	Errors
GET	/api/v1/audit-logs	Read only audit logs, filterable	FR-7.1	200	400

Notes:

GET /api/v1/records

Query params:

status optional, one of the allowed status values

q optional, case-insensitive partial match on company or source

sort_by optional, default last_contacted_date

order optional, asc | desc, default desc

follow_up_due optional, true returns only records due today or earlier

page optional, default 1

page_size optional, one of 10 | 20 | 50 | 100, default 20

Response body:

```
{
  "items": [ ...records... ],
  "page": 1,
  "page_size": 20,
  "total": 137,
  "total_pages": 7
}
```

Errors: 400 if page_size isn't one of the allowed values, or page < 1

3.2 Request/response contract notes

Topic	Decision	Why
Versioning	/api/v1 prefix	Project is still in v1 and to create standard
Naming	Plural nouns, snake_case in paths, params and JSON	Follows PEP 8 (NFR-8) and Pydantic defaults
IDs	Integer, auto increment	Single user and easier to setup for v1
Dates	Date field (last_contacted_date and follow_up_date) follows YYYY-MM-DD; created_at and updated_at as ISO 8601 UTC with Z	A date field is not a moment in time, so it shouldn't carry a timezone
Update semantics	PATCH only	Users edit one field at a time, and PUT would force sending everything
Success codes	POST 201 with body, GET 200, PATCH 200 with updated object, DELETE 204 with no body	One rule per method. PATCH returns the object so the UI doesn't need a second GET
Error shape	{"detail": "..."} for 404 and 400, and keep Django's default 400 body for validation	Zero custom code, and the client handles one top-level key
Error codes	Document only 404, and 400 (import). 405 and 500 are framework or infrastructure concerns, not per-endpoint design	Every listed code should be something your handler deliberately produces
Pagination	Page-number based (page, page_size), not cursor-based. Allowed sizes: 10, 20, 50,	Cursor pagination solves problems you don't have (high write concurrency, huge

	100. Response wraps items in an envelope with page, page_size, total, total_pages	datasets). Page number is simpler to implement and to page through in a UI with numbered buttons
Auth / CORS	None (NFR-6). CORS allows only the Next.js origin (e.g., http://localhost:3000)	Stated explicitly so it's a decision, not an omission
Caching	Server-side, in-process dict, TTL 30s, full invalidation on any record/note write. No HTTP cache headers (Cache-Control/ETag) in v1	Keeps cache logic entirely server-side and simple to reason about; avoids conflating two different caching mechanisms

4. Estimation

4.1 Scale assumptions (even at trivial scale)

Dimension	Assumption	Basis
Users	1	Single user v1
Records (total)	~540 over 6 months	3/day average (realistic range 1–5/day) × 180 days
Records (total, ceiling)	1,800 over 6 months	10/day stretch goal × 180 days
Records (growth rate)	~3/day (~90/month) steady state	Anchor metric
Notes per record	3	Initial, ongoing and conclusion
Writes per record (lifetime)	~6	1 create + 2 edits + 3 notes
Request per day	~38 (~18 writes + ~20 reads)	Derived from records/day × lifetime writes, and estimated session count
Read:write ratio	~1:1	Recomputed, not assumed
Peak concurrency	1	Single user v1
Data size (rough)	~972 KB (~0.95 MB) (~97.2 KB + ~291.6 KB + ~583.2 KB)	~180 bytes per row for records (~540) and notes (~540x3=~1620) and audit logs (~540x6=~3240)

5. Data Model

5.1 Schema

```
Table records {
  record_id integer [pk, not null, increment]
  company string [not null]
  status status [not null]
  company_contact_no string
```

```
  company_email email
  last_date_contacted date
  technical_audit string [note: "Free form text for the audit report summary"]
  source source [note: "Application where the client data comes from"]
  created_at timestamp [not null, default: `now()`]
  modified_at timestamp
}

Table notes {
  note_id integer [pk, not null, increment]
  record_id integer [not null]
  content string
  created_at timestamp [not null, default: `now()`]
  modified_at timestamp
}

Table follow_ups {
  follow_up_id integer [pk, not null, increment]
  record_id integer [not null, unique]
  content string
  follow_up_date date
  created_at timestamp [not null, default: `now()`]
  modified_at timestamp
}

Table audit_logs {
  log_id integer [pk, not null, increment]
  entity_type string [not null, note: "'record' | 'note' | 'follow_up' — always explicit, no
default"]
  entity_id integer [not null, note: "No FK on purpose — must survive deletion of the parent
entity (NFR-3)"]
  action string [not null, note: "'create' | 'update' | 'delete'"]
  initiator string [not null, default: "system", note: "Placeholder until v2 auth adds real
users"]
  content string
  created_at timestamp [not null, default: `now()`]
}

Ref: notes.record_id > records.record_id [delete: cascade]
```

```
Ref: follow_ups.record_id - records.record_id [delete: cascade]


enum status {
  contacted
  ongoing
  completion
}


enum source {
  reddit
  discord
  facebook
  linkedin
  others
}
```



5.2 Decision: fixed follow-up columns vs. separate `followups` table

ADR-0001: Separate `follow_ups` table vs. fixed columns on `records`

Status: Accepted

Context

FR-4.1 describes one follow-up per record, and the relationship is one-to-one. This made fixed columns a legitimate option worth seriously weighing, not table-by-default. The deciding question was whether that one-to-one assumption is likely to hold beyond v1.

Decision

Decided to pick follow ups as another table.

Alternatives considered

Option A: Fixed columns on `records`

- Pros: Simplicity, easy access from records without join

- Cons: No extra metadata and not normalized

Option B: Separate `follow_ups` table

- Pros: Normalized, cleaner look, proper standard.

- Cons: Extra table and extra complexity for joins

Rationale

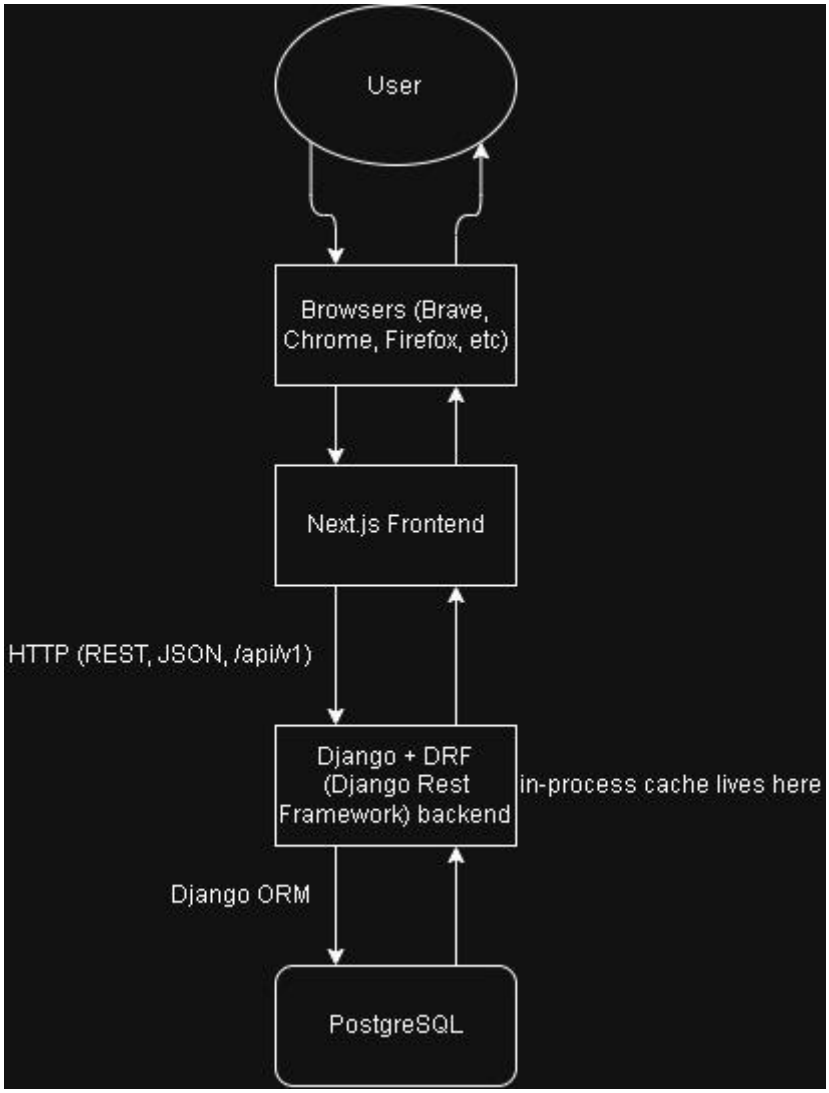
Option A (fixed columns) is the simpler and more appropriate choice for v1's stated requirement of one follow-up per record. However, if a future version needs to support multiple follow-ups per record, Option B avoids a breaking migration — the table already models the one-to-many shape, so relaxing ref: - to ref: > requires no data restructuring. Option B is chosen as a deliberate trade: one extra join today, to avoid a schema rewrite later."

Consequences

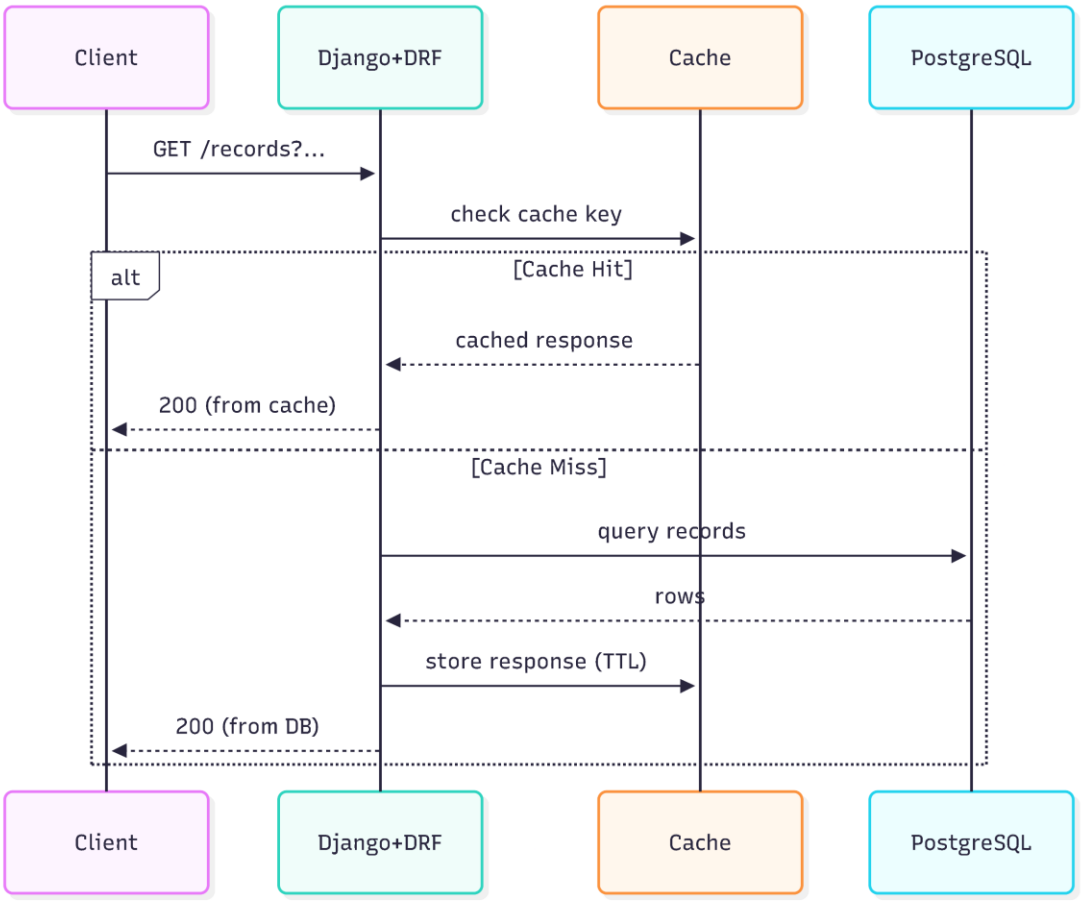
This choice adds a join on every GET /records/{id} and an extra table to include in cascade-delete rules, compared to Option A. In exchange, if a future version needs multiple follow-ups per record, the change is a one-line relationship update (ref: - → ref: >) rather than a breaking schema migration. Revisit this decision if a future requirement needs a follow-up history rather than a single reminder per record.

6. High-Level Design

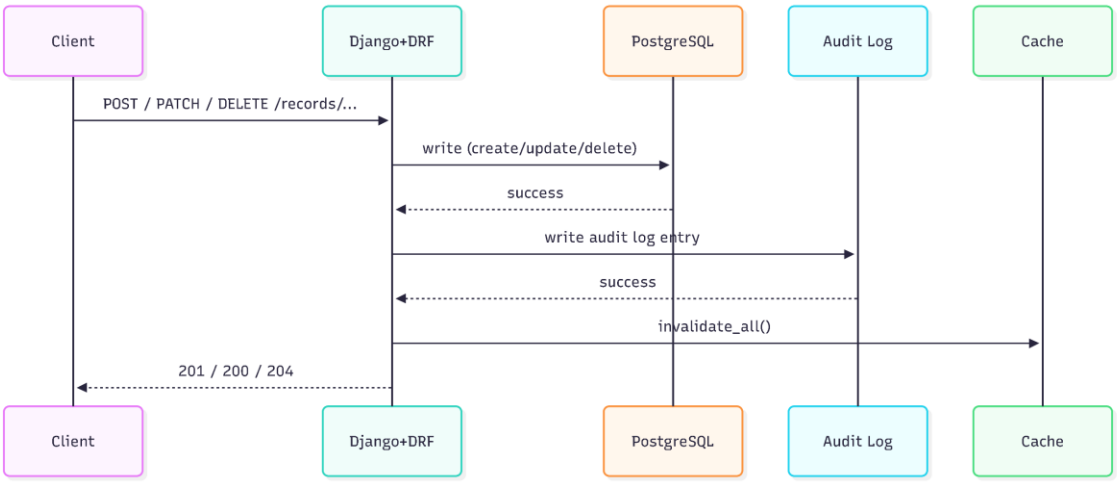
6.1 Architecture diagram



PostgreSQL and the Django backend run together via a single docker-compose.yml (two services, shared network) for local development — environment isolation and easy teardown/rebuild. The Next.js frontend runs natively (npm run dev); containerization offers no benefit for the frontend dev workflow and can slow hot-reload.



Read (Cache Hit / Miss) Flow



Write (DB Write →Audit Log →Cache Eviction) Flow

6.2 Decision: stack rationale, if any alternative was seriously considered

ADR-0002: PostgreSQL over SQLite / MySQL-MariaDB

Status: Accepted

Context

Options are either PostgresSQL, SQLite, MySQL-MariaDB. These are the options mainly due to developer familiarity and most common relational database. Relational database is mainly picked due to the fact, the project is simple as CRUD and we are using structured models. NoSQL was not considered, since the data is structured and relational by nature (records, notes, follow_ups, audit_logs with clear relationships)

Decision

PostgreSQL

Alternatives considered

Option A: SQLite

- Pros: Zero configuration, single file, no separate server process to run or manage. Fully sufficient for this project's data volume (~0.95MB at 6-month scale, per section 4). Simplest possible local development setup.

- Cons: Not commonly used in production/team environments or listed as a target skill for the roles this project is meant to build experience toward. Weaker support for concurrent writes if the project ever grows past single-user (v2 consideration, not a v1 blocker).

Option B: MySQL / MariaDB

- Pros: Also widely used in industry, well-documented, broad hosting support.

- Cons: Historically weaker native support for enum types and case-insensitive text matching compared to Postgres — both used directly in this project's schema (status/source enums, citext-style email handling in section 5). Would require workarounds (e.g., VARCHAR + CHECK constraints) to achieve what Postgres supports natively.

Option C: PostgreSQL

- Pros: Native enum types and citext, both already used in the schema design. Strong standards compliance and widely considered a modern default for new backend projects. Directly relevant to the developer's target roles (e.g., DOST postings).

- Cons: Requires a running database server even in local development (via Docker or a local install), unlike SQLite's zero-setup file-based approach.

Rationale

PostgreSQL was chosen primarily to align with the project's goal of building experience with a database commonly required in target roles (e.g., DOST postings). This is a deliberate trade: SQLite would technically be sufficient, and arguably simpler, for this project's actual scale (single user, <1MB of data per section 4). Postgres is chosen anyway because the skill-building goal outweighs the simplicity SQLite would offer at this scale.

This decision is also supported technically, not just by career goals: the schema (section 5) already uses native enum types (status, source) and case-insensitive email handling, both of which Postgres supports natively and more cleanly than MySQL/MariaDB would.

Consequences

This would be revisited in case a change or update in requirements will be used like adding media, large files, in which Supabase will be used as both storage and database.

ADR-0003: Django + DRF over FastAPI

Status: Accepted

Context

The original plan was built around FastAPI wth the Pydantic schema and SQLAlchemy/Alembic for the data layer. This was mainly the first choice due to familiarity and simplicity of the project. This was then revisited for the same reason as ADR-0002: the developer is building experience with tools that are commonly required in target roles and Django is more frequently requested than FastAPI. This decision was made after the API contract and data model was already designed and alternatives were evaluated to the existing (Alembic vs Django migration)

Decision

Django + DRF

Alternatives considered

Option A: FastAPI (original plan)

- Pros: Native async support, minimal boilerplate for a pure API, automatic OpenAPI/Swagger docs generation, Pydantic validation built directly into route signatures.

- Cons: Less commonly listed as a target skill for the roles this project is meant to build experience toward. No built-in admin interface, ORM, or auth system — those would need to be separately chosen and wired in if the project grows.

Option B: Plain Django (no DRF)

- Pros: One framework to learn instead of two layered together; full visibility into the request/response cycle without a library abstracting it.

- Cons: No built-in request validation, pagination, or structured error formatting — all of which section 3.1/3.2 already assumes (pagination envelope, PATCH semantics, 422-equivalent error responses). Would require hand-building functionality that already exists in a standard library, for no added learning value specific to this project's stated goals.

Option C: Django + DRF

- Pros: Serializers (validation + response shaping) map directly onto the role Pydantic schemas were playing in the original design. Built-in pagination and generic views closely match the contract already designed in section 3. DRF is the de facto standard for building REST APIs in Django, so it's the realistic skill pairing for Django-based roles.

- Cons: Adds a second library's conventions (Serializer, ViewSet, generics classes) on top of Django itself — more to learn than Option A or B individually. More setup/boilerplate (settings, app structure, routers) than FastAPI's minimalism, for a project at this scale.

Rationale

Django + DRF was chosen primarily for the same career-relevance reason as PostgreSQL (ADR-0002): it is the stack most consistent with target roles. This is reinforced by a concrete technical fit: the API contract in section 3.1/3.2 — pagination envelope, PATCH- based partial updates, structured validation errors — was already designed in a shape that matches DRF's own built-in defaults, before DRF was chosen. This suggests the contract and the tool are well matched, rather than the tool being forced onto an unrelated design.

Consequences

This choice gives up FastAPI's native async support and lighter footprint, trading them for Django's batteries-included ecosystem (admin panel, auth system, ORM, migrations) and closer alignment with target job postings. It also means learning DRF's conventions in addition to

Django's own, which FastAPI + Pydantic would not have required as a second layer. Revisit this decision if a future version needs high-throughput async I/O or very low per-request latency at scale — conditions under which FastAPI's async-native design would offer a real advantage that Django's (largely synchronous, by default) request handling does not.

7. Low-Level Design

7.1 Django models

7.2 Pydantic schemas

7.3 Django migrations

7.4 Route handlers

8. Bottlenecks & Future Considerations

CDN and reverse-proxy caching (e.g., Nginx/Varnish in front of Django): not needed at v1 scale (single user, concurrency = 1, see section 4). Revisit if the app moves to multi-user with geographically distributed traffic, or if static asset volume grows enough that Next.js's own deployment-level CDN (e.g., via Vercel) isn't sufficient on its own.

8.1 What breaks at multi-user scale

8.2 v2 candidates (auth, concurrent writes, data isolation)

If this project moves toward team development or production deployment, revisit containerizing the frontend as part of a full docker-compose stack — local dev and deployment have different needs, and this decision applies to local dev only.

9. Testing Plan

9.1 What Pytest covers for v1